Code Explanation

GA Prompt Optimiser for LLM Code Generation

Evolving System Prompts on the APPS Dataset

Group 1

Computational Intelligence — Spring 2026

April 17, 2026

Comprehensive walkthrough of every file in the codebase.
Designed for readers with no prior knowledge of genetic algorithms or LLM APIs.

Contents

1 Introduction **2**
- 1.1 What This Project Does . . . 2
- 1.2 The Codebase at a Glance . . . 2
- 1.3 How to Read This Document . . . 2

2 Configuration: `config.py` **3**
- 2.1 Purpose . . . 3
- 2.2 Full Source Code . . . 3
- 2.3 Block-by-Block Explanation . . . 4
  - 2.3.1 Paths (lines 10–12) . . . 4
  - 2.3.2 Dataset (lines 15–17) . . . 4
  - 2.3.3 Genetic Algorithm (lines 20–27) . . . 4
  - 2.3.4 LLM (lines 30–36) . . . 4
  - 2.3.5 Execution (line 39) . . . 5

3 Seed Prompts: `seeds.py` **6**
- 3.1 Purpose . . . 6
- 3.2 Full Source Code . . . 6
- 3.3 Explanation . . . 6

4 Dataset Loading: `dataset.py` **8**
- 4.1 Purpose . . . 8
- 4.2 Full Source Code . . . 8
- 4.3 Block-by-Block Explanation . . . 9
  - 4.3.1 Why Codewars only? . . . 9
  - 4.3.2 The filter chain (lines 28–33) . . . 9
  - 4.3.3 The split function . . . 9

5 Code Evaluation: `evaluator.py` **10**
- 5.1 Purpose . . . 10
- 5.2 Full Source Code . . . 10
- 5.3 Block-by-Block Explanation . . . 11
  - 5.3.1 Code extraction (lines 7–19) . . . 11
  - 5.3.2 The test harness template (lines 26–42) . . . 11
  - 5.3.3 Sentinel lines (`__R__:`) . . . 12
  - 5.3.4 Partial credit . . . 12

6 Cost Tracking: `cost_tracker.py` **13**
- 6.1 Purpose . . . 13
- 6.2 Key Components . . . 13
  - 6.2.1 CallRecord . . . 13
  - 6.2.2 CostTracker . . . 13

7 LLM Interface: `llm.py` **14**
- 7.1 Purpose . . . 14
- 7.2 The _call function . . . 14
- 7.3 Code Generation . . . 15
- 7.4 Crossover . . . 15

7.5 Mutation Operators 15

8 The Genetic Algorithm: `ga.py` **16**
8.1 Purpose 16
8.2 Individual 16
8.3 Fitness Evaluation 16
8.4 Tournament Selection 17
8.5 Breeding (Crossover + Mutation) 17
8.6 The Main Loop 18

9 Entry Point: `main.py` **19**
9.1 Purpose 19
9.2 Key Steps 19
9.3 Checkpointing 19
9.4 Output Files 19

Glossary **20**

Chapter 1

Introduction

1.1 What This Project Does

This project uses a **Genetic Algorithm (GA)** to automatically discover the best "system prompt" for a Large Language Model (LLM) that generates Python code. A *system prompt* is a paragraph of English instructions that tells the LLM *how* to approach a coding problem (e.g., "think about edge cases first", "identify the algorithm pattern").

The GA treats each system prompt as a **chromosome** — a candidate solution that can be scored, bred, and mutated just like organisms in biological evolution. Over 20 generations, the GA evolves a population of 15 prompts, keeping the best-performing ones and combining their strategies, until it converges on a prompt that solves the most coding problems correctly.

Key Idea

Manual prompt engineering is trial-and-error. This project automates it: the GA *searches* the space of all possible English instruction prompts, using test-case pass rates as the fitness signal.

1.2 The Codebase at a Glance

The project consists of 8 Python files totalling ∼1,000 lines:

File	Purpose	Lines
`config.py`	All settings and hyperparameters	58
`seeds.py`	15 diverse starting prompts	116
`dataset.py`	Load and split APPS problems	88
`evaluator.py`	Execute code, measure correctness	122
`cost_tracker.py`	Track API costs per call	140
`llm.py`	All Claude API interactions	190
`ga.py`	The genetic algorithm loop	228
`main.py`	Entry point, save results	160

Table 1.1: Project file summary.

1.3 How to Read This Document

Each chapter covers one file. Within each chapter:

- A **plain-English summary** of what the file does and why.
- The **actual source code** with syntax highlighting.
- **Block-by-block explanations** that introduce terms progressively.
- **Concept boxes** (blue) for important ideas you'll see again later.

Chapter 2

Configuration: `config.py`

2.1 Purpose

Every number that controls the project's behaviour lives in this single file: how many prompts to evolve, how many generations to run, which Claude model to use, where the dataset is, and so on. No other file contains hard-coded settings. This makes it easy to tweak the experiment without hunting through multiple files.

2.2 Full Source Code

Listing 2.1: `config.py` — All hyperparameters and paths.

```python
"""
config.py -- All hyperparameters and paths in one place.
Edit here; everything else reads from this module.
"""

import os
from pathlib import Path

# Paths
ROOT          = Path(__file__).parent.parent
APPS_TRAIN    = ROOT / "Dataset Examples" / "APPS" / "train"
RESULTS_DIR   = Path(__file__).parent / "results"

# Dataset
EVOLUTION_SIZE = 100    # fixed pool for fitness evaluation during GA
HOLDOUT_SIZE   = 200    # held-out for final within-distribution test
RANDOM_SEED    = 42

# Genetic Algorithm
POPULATION_SIZE    = 15
GENERATIONS        = 20
ELITISM_COUNT      = 2
TOURNAMENT_SIZE    = 3
CROSSOVER_RATE     = 0.8
MUT_INJECT_PROB    = 0.25
MUT_DELETE_PROB    = 0.15
MUT_REPHRASE_PROB  = 0.10

# LLM
ANTHROPIC_API_KEY      = os.environ.get("ANTHROPIC_API_KEY", "")
TARGET_MODEL           = "claude-haiku-4-5-20251001"
TARGET_TEMPERATURE     = 0.0
OPTIMIZER_MODEL        = "claude-sonnet-4-6"
OPTIMIZER_TEMPERATURE  = 0.7
MAX_CODE_TOKENS        = 1024
MAX_OPERATOR_TOKENS    = 600

```

```
38 # Code execution
39 EXEC_TIMEOUT = 5
```

2.3 Block-by-Block Explanation

2.3.1 Paths (lines 10–12)

`ROOT` points to the project's top-level folder (one level above `Code/`). `APPS_TRAIN` points to the APPS dataset's `train/` directory. `RESULTS_DIR` is where output files (best prompts, convergence history) are saved.

Path objects

Python's `pathlib.Path` is a modern way to handle file paths. The `/` operator joins path segments: `ROOT / "APPS" / "train"` produces a full path like `/home/.../APPS/train`. It works on Windows, Mac, and Linux without worrying about slashes.

2.3.2 Dataset (lines 15–17)

`EVOLUTION_SIZE = 100` means the GA evaluates each prompt on 100 fixed coding problems during evolution. `HOLDOUT_SIZE = 200` means 200 *different* problems are reserved for testing the best prompt after evolution finishes — this checks that the prompt didn't just get lucky on the 100 it trained on. `RANDOM_SEED = 42` makes the problem selection reproducible.

2.3.3 Genetic Algorithm (lines 20–27)

- `POPULATION_SIZE = 15` — how many candidate prompts exist at any time.
- `GENERATIONS = 20` — how many breeding cycles the GA runs.
- `ELITISM_COUNT = 2` — the top 2 prompts survive unchanged into the next generation.
- `TOURNAMENT_SIZE = 3` — when selecting a parent, pick 3 random prompts and take the best.
- `CROSSOVER_RATE = 0.8` — 80% chance that two parents are merged (otherwise one is copied).
- `MUT_INJECT_PROB = 0.25` — 25% chance a new strategy sentence is added.
- `MUT_DELETE_PROB = 0.15` — 15% chance a sentence is removed.
- `MUT_REPHRASE_PROB = 0.10` — 10% chance the prompt is reworded.

Elitism

Without elitism, the best prompt could be lost if all its offspring happen to be worse. Elitism copies the top k individuals directly into the next generation, guaranteeing the population never regresses.

2.3.4 LLM (lines 30–36)

Two different Claude models are used:

- **Target model** (`Haiku 4.5`): generates code. This is the model being *optimised* — we're searching for the prompt that makes *this* model write the best code. Temperature 0.0 makes output deterministic so the same prompt always produces the same code on the same problem.
- **Optimizer model** (`Sonnet 4.6`): performs crossover and mutation. It needs creativity (merging prompts, inventing new strategies), so it runs at temperature 0.7.

2.3.5 Execution (line 39)

`EXEC_TIMEOUT = 5`: each generated code snippet gets 5 seconds to run all test cases. If it hangs (infinite loop), the subprocess is killed and the problem scores 0.

Chapter 3

Seed Prompts: `seeds.py`

3.1 Purpose

The GA needs a starting population. This file defines 15 diverse English instruction prompts — the "generation zero" chromosomes. Each one emphasises a different coding strategy so the GA has a broad search starting point.

Why diversity matters

If all 15 seeds said roughly the same thing, crossover would produce near-copies and mutation would be the only source of novelty. Diverse seeds let crossover combine genuinely different strategies: "edge-case analysis" from one parent + "algorithm-first thinking" from another = a potentially better hybrid.

3.2 Full Source Code

Listing 3.1: `seeds.py` — 15 diverse starting prompts (abbreviated).

```python
SEED_PROMPTS: list[str] = [
    # 1 -- Structured analysis
    "You are an expert Python programmer. Before writing code, "
    "analyse the problem: identify the input types, the expected "
    "output type, and any edge cases ...",

    # 2 -- Competitive-programmer mindset
    "Think like a competitive programmer. Identify the algorithm "
    "pattern (math, string manipulation, recursion, sorting, "
    "hashing). Write the most direct, correct solution ...",

    # ... 13 more prompts covering: return-type focus,
    # step-by-step reasoning, example-driven, minimal,
    # edge-case defender, algorithm-first, output-format guard,
    # pseudocode-then-code, constraint-aware, defensive,
    # readability-first, verify-against-examples, direct.
]

assert len(SEED_PROMPTS) == 15
```

3.3 Explanation

The file is a simple Python list of 15 strings, each 2–4 sentences long. The `assert` at the bottom ensures the count matches `POPULATION_SIZE` in `config.py`. If you add or remove a seed, the program will crash immediately with a clear error rather than silently misbehaving.

Each seed targets a different strategy: structured analysis, competitive-programming mindset, return-type focus, step-by-step reasoning, example-driven, minimal, edge-case defence, algorithm-

first, output-format guard, pseudocode-then-code, constraint-aware, defensive/robust, readability-first, verify-against-examples, and direct/focused.

Chapter 4

Dataset Loading: `dataset.py`

4.1 Purpose

This file loads coding problems from the APPS dataset and splits them into three non-overlapping groups: the **evolution pool** (used during GA training), the **held-out set** (used for final evaluation), and the **rest** (unused buffer).

4.2 Full Source Code

Listing 4.1: `dataset.py` — Load and split APPS problems.

```python
import json
import random
import sys
from pathlib import Path


def load_problems(apps_train_dir):
    """Return every Codewars introductory fn_name problem."""
    sys.set_int_max_str_digits(1_000_000)
    problems = []
    train_dir = Path(apps_train_dir)

    for prob_dir in train_dir.iterdir():
        meta_f = prob_dir / "metadata.json"
        io_f   = prob_dir / "input_output.json"
        q_f    = prob_dir / "question.txt"

        if not (meta_f.exists() and io_f.exists()
                and q_f.exists()):
            continue
        try:
            meta = json.loads(meta_f.read_text())
            io   = json.loads(io_f.read_text())
        except Exception:
            continue

        if (meta.get("difficulty") != "introductory"
                or "fn_name" not in io
                or "codewars.com" not in meta.get("url", "")
                or not io.get("inputs")
                or not io.get("outputs")):
            continue

        problems.append({
            "id":       prob_dir.name,
            "question": q_f.read_text().strip(),
            "fn_name":  io["fn_name"],
            "inputs":   io["inputs"],
```

```
39                "outputs":  io["outputs"],
40            })
41    return problems
42
43
44 def split(problems, evolution_size, holdout_size,
45          seed=42):
46    """Split into evolution, holdout, and rest."""
47    rng = random.Random(seed)
48    shuffled = problems.copy()
49    rng.shuffle(shuffled)
50    evolution = shuffled[:evolution_size]
51    holdout   = shuffled[evolution_size:
52                         evolution_size + holdout_size]
53    rest      = shuffled[evolution_size + holdout_size:]
54    return evolution, holdout, rest
```

4.3 Block-by-Block Explanation

4.3.1 Why Codewars only?

The APPS dataset contains problems from multiple sources: Codewars, LeetCode, Codeforces, etc. This project filters to *Codewars introductory problems with a* `fn_name` field for three reasons:

1. **Consistent execution**: Codewars problems define a standalone function (e.g., `def grader(score)`). LeetCode problems use `class Solution` with methods, which complicates extraction.
2. **Consistent I/O format**: Codewars stores inputs as argument lists and outputs as `[return_value]` (wrapped in a single-element list). LeetCode uses raw values and sometimes string-encoded arguments.
3. **Quantity**: 2,105 problems pass the filter — far more than the 100 + 200 needed.

4.3.2 The filter chain (lines 28–33)

Five conditions must all be true:

- Difficulty is "introductory" (not interview or competition).
- The problem has a `fn_name` field (function-call style, not stdin/stdout).
- The URL contains "codewars.com".
- Both `inputs` and `outputs` are non-empty.

4.3.3 The split function

A deterministic shuffle (using a fixed seed) ensures that *every run uses the same 100 problems* for fitness evaluation. This is critical: if the problems changed between generations, fitness scores would not be comparable.

Train/test split

Just like in machine learning, you never evaluate on the data you trained on. The GA "trains" on the 100-problem evolution pool; the 200-problem held-out set measures whether the evolved prompt generalises to unseen problems.

Chapter 5

Code Evaluation: `evaluator.py`

5.1 Purpose

This file does two things: (1) extract Python code from an LLM's text response, and (2) execute that code against test cases in a sandboxed subprocess, returning a partial-credit fitness score.

5.2 Full Source Code

Listing 5.1: `evaluator.py` — Code extraction and test execution.

```python
import re
import subprocess
import sys

# --- Code extraction ---

def extract_code(response):
    """Pull Python code from an LLM response."""
    m = re.search(r"```python\s*\n([\s\S]*?)```", response)
    if m:
        return m.group(1).strip()
    m = re.search(r"```\s*\n?([\s\S]*?)```", response)
    if m:
        return m.group(1).strip()
    return response.strip()


def has_function(code, fn_name):
    """True if def fn_name( appears in the code."""
    return bool(
        re.search(rf"\bdef\s+{re.escape(fn_name)}\s*\(",
                  code))


# --- Subprocess test runner ---

_HARNESS_TEMPLATE = """\
import sys
sys.set_int_max_str_digits(1_000_000)

{code}

def __run__(inp, expected):
    try:
        result = {fn_name}(*inp)
        tag = "1" if [result] == expected else "0"
        sys.stdout.write("__R__:" + tag + "\\n")
        sys.stdout.flush()
    except Exception:
```

```
40          sys.stdout.write("__R__:0\\n")
41          sys.stdout.flush()
42
43  {calls}
44  """
45
46
47  def _build_script(code, fn_name, inputs, outputs):
48      call_lines = "\n".join(
49          f"__run__({repr(inp)}, {repr(out)})"
50          for inp, out in zip(inputs, outputs)
51      )
52      return _HARNESS_TEMPLATE.format(
53          code=code, fn_name=fn_name, calls=call_lines)
54
55
56  def run_tests(code, fn_name, inputs, outputs,
57                timeout=5):
58      """Execute code against test cases.
59      Returns partial-credit: passed / total."""
60      if not code or not has_function(code, fn_name):
61          return 0.0
62
63      script = _build_script(code, fn_name,
64                             inputs, outputs)
65      try:
66          proc = subprocess.run(
67              [sys.executable, "-c", script],
68              capture_output=True, text=True,
69              timeout=timeout)
70          results = [
71              int(line[len("__R__:"):])
72              for line in proc.stdout.splitlines()
73              if line.startswith("__R__:")]
74          if not results:
75              return 0.0
76          return sum(results) / len(inputs)
77      except subprocess.TimeoutExpired:
78          return 0.0
79      except Exception:
80          return 0.0
```

5.3 Block-by-Block Explanation

5.3.1 Code extraction (lines 7–19)

LLMs often wrap code in Markdown fences like "‘python ... "‘. The `extract_code` function tries three strategies in order:

1. Look for a "‘python fenced block.
2. Look for any "‘ fenced block.
3. Give up and treat the entire response as code.

5.3.2 The test harness template (lines 26–42)

Instead of importing the generated code into our process (which could crash the entire GA), we run it in a **subprocess** — a completely separate Python process. The template builds a

standalone script that:

1. Includes the generated code verbatim.
2. Defines a `__run__` function that calls `fn_name(*inp)`, compares `[result]` with `expected`, and prints a sentinel line `__R__:1` or `__R__:0`.
3. Appends one `__run__(...)` call per test case.

Why `[result] == expected`?

Codewars APPS problems store outputs as `[return_value]` — the result wrapped in a single-element list. So if a function returns `5`, the expected output is `[5]`. Wrapping the result in `[result]` makes the comparison match this format.

5.3.3 Sentinel lines (`__R__:`)

If the generated code contains stray `print()` statements, they would mix into stdout and corrupt our results. The `__R__:` prefix is a sentinel: when parsing the output, we only look at lines starting with this prefix and ignore everything else.

5.3.4 Partial credit

The function returns `sum(results) / len(inputs)` — the fraction of test cases passed. This is critical for the GA: if a prompt produces code that passes 7 out of 10 tests, it scores 0.7, not 0.0. This gives the GA a smooth gradient to optimise, rather than a binary "all or nothing" signal.

Why not binary pass@1?

With only binary scoring, a prompt that solves 0 test cases and one that solves 9 out of 10 would both score 0. The GA couldn't tell them apart. Partial credit is used *during evolution* for better search; the final reported metric is still standard pass@1.

Chapter 6

Cost Tracking: cost_tracker.py

6.1 Purpose

Every API call costs real money. This module logs the exact token count of every call, computes the dollar cost using official Anthropic pricing, and saves a detailed JSON breakdown to the `Costs/` folder after each run.

6.2 Key Components

Listing 6.1: `cost_tracker.py` — Pricing constants (excerpt).

```
PRICING = {
    "claude-haiku-4-5-20251001": (1.00,  5.00),
    "claude-sonnet-4-6":         (3.00, 15.00),
    # Batch API (50% discount)
    "claude-haiku-4-5-20251001:batch": (0.50, 2.50),
    "claude-sonnet-4-6:batch":         (1.50, 7.50),
}
```

Each tuple is `(input_$/MTok, output_$/MTok)`. These are the official prices from the Anthropic pricing page (April 2026). MTok = million tokens.

6.2.1 CallRecord

Each API call creates a `CallRecord` with four fields: `phase` (e.g., "fitness", "crossover"), `model`, `input_tokens`, and `output_tokens`. Cost is computed on the fly:

Listing 6.2: Cost calculation per call.

```
@property
def total_cost(self):
    rate_in, rate_out = PRICING[self.model]
    return (self.input_tokens * rate_in
            + self.output_tokens * rate_out) / 1_000_000
```

6.2.2 CostTracker

The `CostTracker` class accumulates all `CallRecord`s and provides:

- `record(phase, model, in_tokens, out_tokens)` — log one call.
- `_by_phase()` — aggregate by GA phase (fitness, crossover, etc.).
- `_by_model()` — aggregate by Claude model.
- `flush()` — write a timestamped JSON to `../Costs/`.

A module-level singleton `tracker = CostTracker()` is imported by `llm.py` so every API call is automatically logged.

Chapter 7

LLM Interface: `llm.py`

7.1 Purpose

This file is the sole interface to the Claude API. It provides five public functions:

1. `generate_code` — ask the target LLM to solve a problem (fitness evaluation).
2. `crossover` — merge two parent prompts into one offspring.
3. `mutate_inject` — add a new strategy sentence.
4. `mutate_delete` — remove the weakest sentence.
5. `mutate_rephrase` — reword without changing meaning.

Every call goes through a private `_call` function that handles retries and cost logging.

7.2 The _call function

Listing 7.1: `llm.py` — Core API wrapper (simplified).

```python
def _call(phase, model, temperature, max_tokens,
          system, user, retries=3):
    client = _get_client()
    for attempt in range(retries):
        try:
            resp = client.messages.create(
                model=model,
                max_tokens=max_tokens,
                temperature=temperature,
                system=system,
                messages=[{"role": "user",
                           "content": user}])
            # Log tokens to cost tracker
            tracker.record(
                phase=phase, model=model,
                input_tokens=resp.usage.input_tokens,
                output_tokens=resp.usage.output_tokens)
            return resp.content[0].text
        except anthropic.RateLimitError:
            time.sleep(2 ** attempt * 5)
        except anthropic.APIError as e:
            if attempt == retries - 1:
                return ""
            time.sleep(2)
    return ""
```

Retry with exponential backoff

If the API returns a rate-limit error, the code waits 5s, then 10s, then 20s before retrying. This "exponential backoff" pattern prevents hammering a busy server and is standard practice for API integrations.

7.3 Code Generation

Listing 7.2: Code generation — the fitness call.

```python
def generate_code(system_prompt, question, fn_name):
    user_msg = (f"{question}\n\n"
                f"Write a Python function named "
                f"`{fn_name}`.")
    return _call(
        phase="fitness",
        model=config.TARGET_MODEL,        # Haiku 4.5
        temperature=0.0,                  # deterministic
        max_tokens=config.MAX_CODE_TOKENS,
        system=system_prompt,             # THE CHROMOSOME
        user=user_msg)
```

The `system_prompt` is the GA chromosome — the prompt being evolved. The `user` message contains the problem description and the required function name. This separation is important: the system prompt is *general* (applies to any problem), while the user message is *problem-specific*.

7.4 Crossover

Listing 7.3: Crossover meta-prompt.

```python
_CROSSOVER_USER = """
Combine these two coding-instruction prompts into one
coherent prompt. Preserve the strongest strategies from
each parent. Do not simply concatenate.

--- Parent A ---
{parent_a}

--- Parent B ---
{parent_b}
"""
```

The optimizer LLM (Sonnet 4.6) receives both parents and must produce a *unified* offspring. The meta-prompt explicitly forbids concatenation — it must blend strategies intelligently.

7.5 Mutation Operators

Three independent mutations, each with its own meta-prompt:

- **Inject** (25%): "Add one new, specific coding strategy to this prompt." — introduces novelty.
- **Delete** (15%): "Remove the sentence that contributes the least." — prevents prompt bloat.
- **Rephrase** (10%): "Rephrase using different words, keep same meaning." — explores wording variations.

These are *independent*: on any given offspring, all three can fire, none can fire, or any combination. This gives $2^3 = 8$ possible operator combinations per offspring.

Chapter 8

The Genetic Algorithm: ga.py

8.1 Purpose

This is the core of the project. It implements the full evolutionary loop: initialise a population, evaluate fitness, select parents, breed offspring, and repeat.

8.2 Individual

Listing 8.1: The Individual dataclass.

```python
@dataclass
class Individual:
    prompt: str
    fitness: float = -1.0
    uid: str = field(
        default_factory=lambda: uuid.uuid4().hex[:8])
    parent_a: str = ""
    parent_b: str = ""
    operators: str = ""
```

Each `Individual` is one chromosome — a system prompt. It carries its fitness score, a unique ID, parent IDs (for genealogy tracking), and which operators were applied.

8.3 Fitness Evaluation

Listing 8.2: Computing fitness for one individual.

```python
def compute_fitness(individual, problems):
    total = 0.0
    for prob in problems:
        response = llm.generate_code(
            system_prompt=individual.prompt,
            question=prob["question"],
            fn_name=prob["fn_name"])
        code = evaluator.extract_code(response)
        score = evaluator.run_tests(
            code=code,
            fn_name=prob["fn_name"],
            inputs=prob["inputs"],
            outputs=prob["outputs"],
            timeout=config.EXEC_TIMEOUT)
        total += score
    return total / len(problems)
```

For each of the 100 problems in the evolution pool:

1. Send the prompt + problem to Claude → get code.
2. Extract the code from the response.

3. Run it against test cases → get partial-credit score.
4. Average all scores → fitness.

8.4 Tournament Selection

Listing 8.3: Tournament selection.

```
def tournament_select(population, rng):
    contestants = rng.sample(population,
                             config.TOURNAMENT_SIZE)
    return max(contestants,
               key=lambda ind: ind.fitness)
```

Tournament selection

Pick 3 random individuals from the population. The fittest one wins and becomes a parent. This is run twice to get two parents for breeding. Tournament selection balances exploitation (the fittest usually wins) with exploration (weaker individuals occasionally participate if they happen to be drawn).

8.5 Breeding (Crossover + Mutation)

Listing 8.4: The breed function.

```
def breed(parent_a, parent_b, rng):
    ops = []
    # Crossover (80% chance)
    if rng.random() < config.CROSSOVER_RATE:
        prompt = llm.crossover(parent_a.prompt,
                               parent_b.prompt)
        ops.append("crossover")
    else:
        prompt = rng.choice(
            [parent_a.prompt, parent_b.prompt])
        ops.append("copy")

    # Mutations (independent)
    if rng.random() < config.MUT_INJECT_PROB:
        prompt = llm.mutate_inject(prompt)
        ops.append("inject")
    if rng.random() < config.MUT_DELETE_PROB:
        prompt = llm.mutate_delete(prompt)
        ops.append("delete")
    if rng.random() < config.MUT_REPHRASE_PROB:
        prompt = llm.mutate_rephrase(prompt)
        ops.append("rephrase")

    return Individual(
        prompt=prompt,
        parent_a=parent_a.uid,
        parent_b=parent_b.uid,
        operators="+".join(ops))
```

8.6 The Main Loop

Listing 8.5: GA main loop (simplified).

```
def run(seed_prompts, problems, on_generation=None):
    rng = random.Random(config.RANDOM_SEED)
    population = [Individual(prompt=p)
                  for p in seed_prompts]

    # Evaluate generation 0
    for ind in population:
        ind.fitness = compute_fitness(ind, problems)

    for gen in range(1, config.GENERATIONS + 1):
        population.sort(key=lambda x: x.fitness,
                        reverse=True)
        # Elitism: keep top 2
        new_pop = population[:config.ELITISM_COUNT]

        # Breed offspring to fill remaining slots
        while len(new_pop) < config.POPULATION_SIZE:
            p1 = tournament_select(population, rng)
            p2 = tournament_select(population, rng)
            child = breed(p1, p2, rng)
            new_pop.append(child)

        # Evaluate new offspring only
        for ind in new_pop[config.ELITISM_COUNT:]:
            ind.fitness = compute_fitness(
                ind, problems)

        population = new_pop

    population.sort(key=lambda x: x.fitness,
                    reverse=True)
    return population[0], history
```

Generational replacement

Each generation, the *entire* population is replaced except for the elite individuals. This is called **generational replacement** (as opposed to steady-state, where only one or two individuals are replaced at a time). It creates strong selection pressure: only the best survive.

Chapter 9

Entry Point: `main.py`

9.1 Purpose

This file orchestrates the full run: load data, run the GA, evaluate on held-out data, save all results, and write the cost report.

9.2 Key Steps

1. **Pre-flight checks**: verify API key and dataset path exist.
2. **Load and split**: call `dataset.load_problems()` and `dataset.split()`.
3. **Create run directory**: `results/run_<timestamp>/` with config snapshot and problem IDs.
4. **Run GA**: call `ga.run()` with a checkpoint callback that saves population state after each generation.
5. **Save results**: `best_prompt.txt`, `history.json`, generation checkpoints.
6. **Holdout evaluation**: run the best prompt on 200 unseen problems.
7. **Cost report**: call `tracker.flush()` to save a JSON to `../Costs/`.

9.3 Checkpointing

After each generation, `save_checkpoint()` writes the full population state (all prompts, fitnesses, parent IDs, operators) to a JSON file. If the run crashes mid-way, you lose at most one generation of work.

9.4 Output Files

File	Contents
`best_prompt.txt`	The evolved prompt (main artifact)
`history.json`	Best/mean/worst fitness per generation
`generation_XX.json`	Full population state (for genealogy)
`summary.json`	Final fitness on evolution + holdout
`config.json`	Exact hyperparameters used
`evolution_ids.json`	Which 100 problem IDs were used
`../Costs/cost_*.json`	Detailed cost breakdown

Table 9.1: Output files produced by each run.

Glossary

Chromosome
A candidate solution in a genetic algorithm. In this project, each chromosome is a natural-language system prompt.

Crossover A genetic operator that combines two parent chromosomes to produce an offspring. Here, the LLM merges two parent prompts into one.

Elitism Preserving the top k individuals unchanged into the next generation, preventing loss of the best solution.

Evolution pool
The fixed set of 100 coding problems used to evaluate fitness during the GA.

Fitness A numeric score measuring how good a chromosome is. Here, fitness = average fraction of test cases passed across 100 problems.

fn_name The function name that the LLM must write. Defined in each APPS problem's `input_output.json`.

Generation One cycle of the GA: select parents, breed offspring, evaluate, replace.

Held-out set
Problems reserved for evaluating the final evolved prompt, never seen during evolution.

Individual One member of the GA population — a prompt paired with its fitness score and metadata.

LLM Large Language Model — a neural network trained on text that can generate code, merge prompts, etc.

MTok Million tokens — the billing unit for LLM APIs.

Mutation A genetic operator that modifies a single chromosome. Three types: inject (add strategy), delete (remove sentence), rephrase (reword).

Partial credit
Scoring a problem as (tests passed / total tests) rather than binary all-or-nothing.

pass@1 The standard code-generation metric: fraction of problems where all test cases pass on a single attempt.

Population The set of all chromosomes alive at a given generation.

Sentinel A unique string prefix (`__R__:`) used to distinguish test results from stray output in subprocess execution.

System prompt
An instruction paragraph sent to the LLM that shapes its behaviour across all messages in a conversation.

Temperature
An LLM parameter controlling randomness. 0 = deterministic; higher = more creative/varied.

Tournament selection
Selecting parents by randomly sampling k individuals and picking the fittest among them.